Computer Security

Riassunto completo del corso

Politecnico di Milano

A.A. 2025-2026, II Semestre

Contents

1 Prefazione **2**

1.1 Disclaimer 2

1.2 Keywords 2

2 Introduction **2**

2.1 Key concepts 2

2.1.1 Example: exploiting a vulnerability 3

2.1.2 Example: A strawman software vulnerability 3

2.2 Assets, Threats, Attacks and Threat Agents 4

2.2.1 What is Protected and what is Secured 4

2.2.2 Assets 4

2.2.3 Threats and Attacks 4

2.3 Risks 4

2.4 Security versus Cost Balance 5

1 Prefazione

1.1 Disclaimer

I contenuti di questo PDF sono stati compilati con il solo scopo di avere una risorsa esaustiva e facilmente consultabile degli argomenti trattati nel corso di Computer Security sostenuto dal professore Zanero: non sono in alcun modo materiale sostitutivo alle lezioni, che frequentate in presenza mi hanno permesso di capire a fondo la materia. Sconsiglio di usare esclusivamente suddetto documento per cercare di prepararsi all'esame, supporto invece l'uso di questa dispensa per fare il punto della situazione durante il periodo di studio.

1.2 Keywords

2 Introduction

How can we define what is a secure system? Usually, it follows the **CIA** paradigm to correctly define what a secure system is, the problem lies inside its definition:

- **Confidentiality** - the system should not leak information to unauthorized parties.
- **Integrity** - the system should not allow unauthorized parties to modify data **and** should not allow authorized parties to modify data in an unauthorized way.
- **Availability** - information should be always available to authorized parties.

The problem lies inside **Integrity**, if deciding whether a party is authorized or not is simple since it's a binary decision, the decision of whether a modification is authorized or not is much more complex and involves a lot of factors. Also, **Availability** is what pays off in the end. What can we consider **secure**? Something is secured if it takes an **effort** to break it: the more effort it takes, the more secure it is.

Secure System

A **Secure System** is a system that requires a lot of effort to be broken.

2.1 Key concepts

We firslty need some definitions useful to understand the world of security:

- **Vulnerability** - It is a shortcut that allows an attacker to break the system with less of the intended effort: it is something that allows to violate one of the CIA properties.
- **Exploit** - It is a specific way to use one or more vulnerabilities to accomplish a specific objective that violates one of the CIA properties.
- **Asset**
- **Threat**
- **Risk**

Understanding what is and what isn't a vulnerability

Not all vulnerabilities lookalikes are actual vulnerabilities or mistakes to exploit: sometimes, something is vulnerable but the accomplishment of the attack doesn't result in a violation of the CIA properties.

2.1.1 Example: exploiting a vulnerability

When exploiting a vulnerability, we firstly need to find the security assumption that lays behind the system: in this example, the thing that ensures that the lock works is the fact that the pins are being pushed down always by the strings of the lock, so the only way to open the lock is to push up all the pins at the same time. Lockpicking is using a vulnerability of the lock to open it: this is using an **exploit**, using something to take advantage of a vulnerability to break the system.

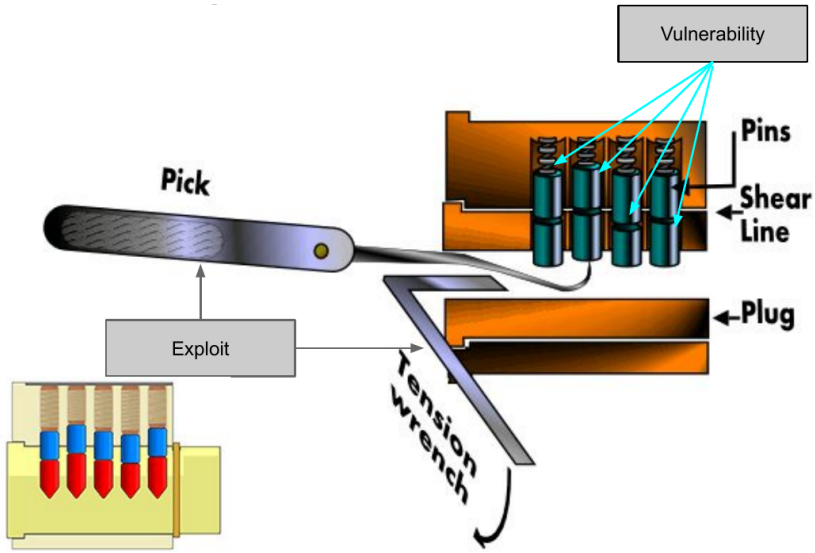

Figure 1: Lockpicking example

How can we resolve this vulnerability? Short answer: we make it harder to exploit it. Oiled locks are harder to exploit because of the deletion of friction, but still are vulnerable: some vulnerabilities are implicit in the design of the system itself.

2.1.2 Example: A strawman software vulnerability

Code

Strawman software vulnerability

```
int i;
unsigned short s;
i = atoi(argv[1]); // parse command line parameter as int
if (i == 0) { // check
    printf("Invalid number: value must be > 0\n");
    return -1;
}
s = i;
if (s == 0) { //security check
    printf("Access GRANTED!\n");
}
```

Implicit casting from int to unsigned short can cause a vulnerability (in this case, to have 0 in the variable **s** even if $\mathbf{i = 65536}$).

2.2 Assets, Threats, Attacks and Threat Agents

2.2.1 What is Protected and what is Secured

It is crucially important that it is different to say that something is **safe** and that something is **protected**: something can be more protected but less safe, because of the presence of specific **threats** or because of its value for the individual threat (a soldier is less protected but safer against a terrorist than the polititian inside the ACV). The Cheyenne Mountain is protected against nuclear attacks, but it is not safe against rattlesnakes.

Safety agains Threats

A system is protected (and safe) always against a specific threat, it is of no sense to talk about the security of a system without considering the threat we are trying to protect against.

Watch Out!

It is total bullshit to talk about protecting a system and potential vulnerabilities without having a clear understanding of the threat models: professionals in the field of security only talk about them, not about vulnerabilities.

2.2.2 Assets

Asset

An **asset** identifies what is valuable for the organization.

We will be focusing mostly on **IT Assets**, like data, hardware and software: identifying assets is key into value them from the point of view of an attacker. Different attackers may have a different value for our different assets, it is our job to understand what could become the value of our resources and the potential loss of our assets.

2.2.3 Threats and Attacks

Threat

A **threat** is a potential violation of a CIA rule and the circumstances that can lead to said violation

A threat is usually caused by an intentional attacks, caused by attack agents (we will not cover accidents, non intentional attacks).

Hakers are not Attackers

a Hacker is someone with an advanced understanding of computers and computer networks, malicious hackers are called **black hats** (from western movies).

2.3 Risks

We just understood that **no system is invulnerable**: there will always be a vulnerability that could become a threat to our system from a specific point of view of an attacker. The only thing we can talk about is **how exposed to damage** is our system: we are talking about **risks**.

Risk

A **risk** is a statistical and economical evaluation of the exposure to damage because of the presence of vulnerabilities and threats.

Risk can be defined as a mathematical formula:

$$\text{Risk} = \text{Asset} \times \text{Vulnerabilities} \times \text{Threats} \tag{1}$$

A big problem is that the Risk term is almost always impossible to compute: we hardly know the value of our assets from an attacker point of view, we have little information on how vulnerable our system is and mostly important we are never able to identify the threats our system will encounter. It is important to note though that Assets and Vulnerabilities are **controllable variables**, the values we can actually manipulate, whereas Threats are **indepentent variables**, completely separated from our knowledge. It becomes clearer and clearer that we need to take decisions: we need to balance the urge of damage containment and the reduction of vulnerabilities to the costs they bring. This is **Security**.

2.4 Security versus Cost Balance

In security we can distinct two different costs:

- **Direct Costs** - Management, operational and equipment
- **Indirect Costs (more important)** - Less usability, slower performance, less privacy, reduced productivity

(remember when no one locked their phones when biometrics was not a thing? it was more the cost of usability than the perceived threat of having the phone opened by someone else!)

The Privacy Tradeoff Problem

When the diminuition of privacy is traded with "more security", since security is composed by privacy, it means something bad is going to happen.